

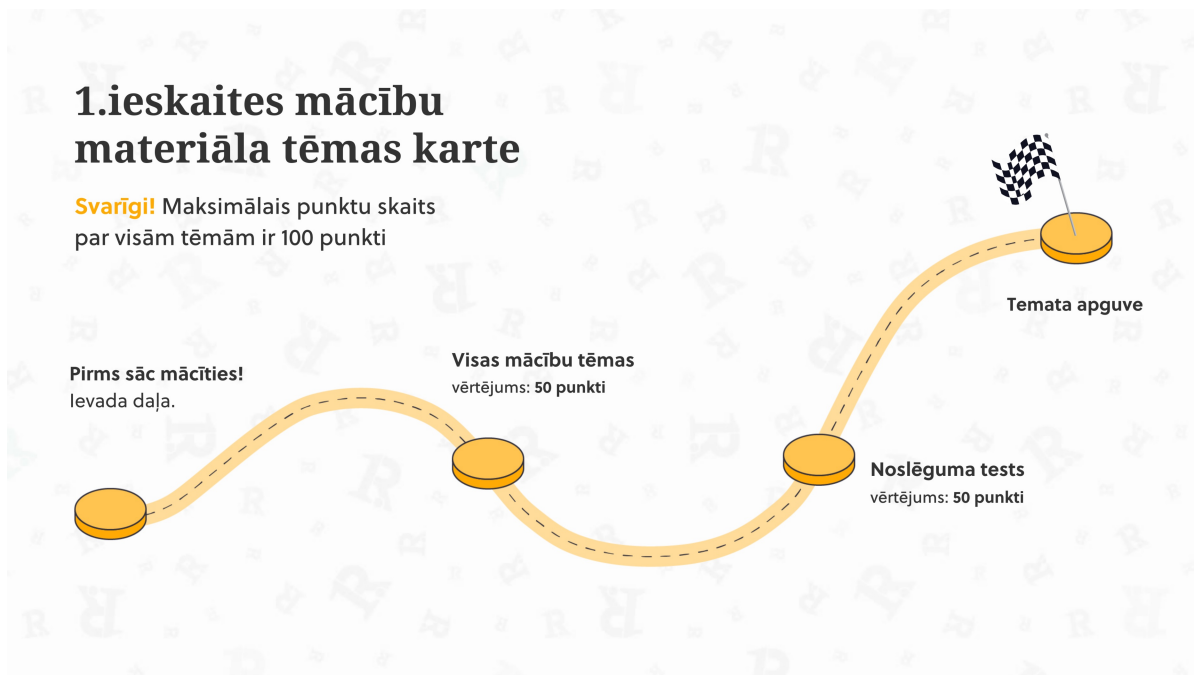
Datorikas 9. klases 1. ieskaites mācību materiāls

Pēdējo reizi atjaunots: 29.08.2025 08:05

Pirms sāc mācīties! Ievada daļa

Esi sveicināts priekšmeta **Datorikas 1.ieskaites mācību materiāla apgūvē. Esmu Rīgas 1. Tālmācības vidusskolas pedagogs Gatis Cielava un tālāk** palīdzēšu spert pirmos soļus, lai Tu apgūtu šo materiālu! **1. ieskaites mācību materiāla nosaukums ir "Javascript (2. daļa no 5)", un tas sastāv no šādām sadaļām:**

- **Pirms sāc mācīties!** Ievada daļa.
- Ievads
- **1. tēma.** Random skaitļu ģenerators
- **2. tēma.** Metode setInterval
- **3. tēma.** Klaviatūras pogu izmantošana
- Mājas darbs Nr.1
- Mājas darbs Nr.2
- Uzdevumu iesniegšana un nosacījumi
- **Noslēguma tests.**
- **Temata apguve.**



👉 **Apgūstot šo mācību materiālu 1. ieskaitei, Tu iemācīsies:**

- 1 Uzzināsi kas ir Random skaitļi un kā tos izmantot
- 2 Iepazīsies ar SetInterval (funkciju izsaukšanai ik pēc noteikta laika)
- 3 Apskatīsi attēla ienešanu un attēlošanu uz Canvas
- 4 Sapratīsi, kā likt Tavam kodam reaģēt uz klaviatūras taustiņu nospiešanu
- 5 Izveidosi 2. mājas darbus, tie būs jāpievieno ieskaitei

🗨️ Ja rodas jautājumi vai neskaidrības, sazinies ar skolotāju vai klasesbiedriem Discord vidē!

Vērtēšanas kārtība

Mācību sasniegumu vērtēšanas kārtības noteikumi izstrādāti saskaņā ar LR likumiem un normatīvajiem dokumentiem. Iepazīties ar vērtēšanas kārtību Tu vari [šeit](#). Arī [šeit](#) var iepazīties ar skolēna mācību sasniegumu vērtēšanu 10 ballu skalā pēc Skola2030 dokumentiem.

Lai pārietu uz ieskaites kārtošānu, ir nepieciešams apgūt 1.ieskaites mācību materiālu, izpildīt pašpārbaudes uzdevumus un noslēguma testu, iegūstot vismaz 60% (xp).

Saņemtie pieredzes punkti (xp) tiek piešķirti par mācību materiāla apguvi un pašpārbaudes uzdevumu izpildi. Pieredzes punkti norāda cik kvalitatīvi esi apguvis mācību materiālu.

Vismaz 60 no 100 punktiem par mācību materiāla apgūšanu

[<](#) KURSI



IESKAITES MĀCĪBU MATERIĀLS

0%



 0 UZDEVUMI

Pēc kursa apgūšanas, varēsi veiksmīgi kārtot ieskaiti.

 525

NEPIECIEŠAMĀIS LAIKS MINŪTĒS



VIEGLS

Ievads

JavaScript turpinājums

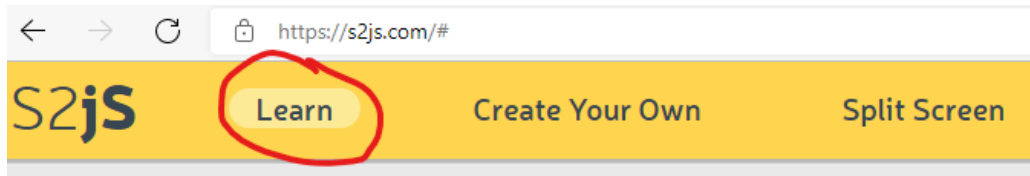
Šajā ieskaitē turpināsim apgūt programmēšanu JavaScript valodā.

Joprojām, lai būtu vienkāršāk un ērtāk, darbu veiksīm mājas lapā <https://s2js.com/>

Mājas lapa ir angļu valodā, bet ja Jums nepietiek zināšanu vai ir kas neskaidrs, varat augšējā labajā stūrī nomainīt valodu. Tas gan būs automātiskais Google tulkotājs, tāpēc vietām tulkojums var būt neprecīzs vai pat smieklīgs.

Mājas lapā ir apmācības zona un mēs kopā iziesim tai cauri.

Šajā mājas lapā Jums ar vecāku palīdzību ir jāreģistrējas, jo tas dod iespēju saglabāt iesāktos darbus un tos pārsūtīt. Apmācības procesā reģistrēšanas nav obligāta.



Atverot šo mājas lapu, augšējā kreisajā stūrī nospiediet **Learn**. Tad aizvērsies šobrīd liekais programmas izveides logs.

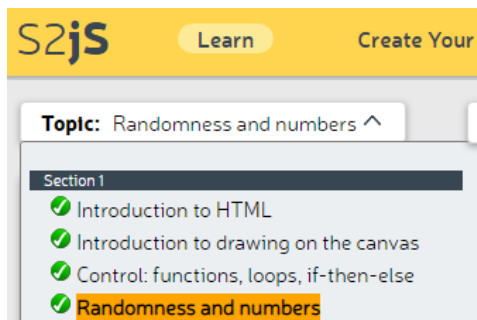


Tālāk zem Topic varat izvēlēties nodaļu, kuru vēlaties apgūt un sadaļā Section var jau izvēlēties šīs nodaļas apakšpunktus. Tas noderēs ja vēlēšities atkārtot kādu konkrētu jautājumu.

Pirmās 3 sadaļas – "Iepazīšanās ar HTML", "Canvas", "Funkcijas un cikli", mēs aplūkojām 8. klasē 4. ieskaitē. Ļoti ieteiktu sākt ar šo sadaļu atkārtošānu.

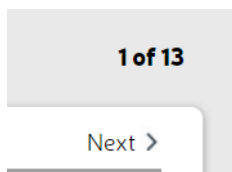
1. Tēma - Random skaitļu generators

Random skaitļu generators



Atveriet sadaļu **Randomness and numbers**.

Lapaspuse 1 no 9.



Stūrītī spiežot **Next**, varam pāriet uz nākamo nodaļas lapaspusi. Atgriezties iepriekšējā lapaspusē varam ar **Previous**.

Kad tiksī līdz mājas darbiem, ievēro, ka mājas darbā daļa no uzdevuma būs šādi krāsaini aplī (attēlā). Tātad kādā brīdī mācību sadaļā būs redzams kods šādiem aplīem un to varēs nokopēt un palabot.

Lapaspuse 2 no 9.

Šeit parādīts kādas zīmes tiek izmantotas matemātiskām darbībām JavaScript valodā:

- + Saskaitīšana
- Atņemšana
- * Reizināšana
- / Dalīšana

Ja vajag kādu no matemātiskajām darbībām veikt pirmo, tad to liekam iekavās.

var area = width * height;//Divu skaitļu reizinājums

var summa = (2 + 3) * 5; // summa ir 25

var summa = 2 + (3 * 5); // summa ir 17

Atceries! Jaunos mainīgos mēs definējam ar **var**

Piemēri:

Turpmāk es parādīšu dažus piemērus par katru tēmu, lai Jūs vieglāk varētu saprast, kas tiek mainīts un ko mēs ar to iegūstam.

P1. var a=1;

Šajā rindiņā tiek definēts mainīgais **a** un tam piešķirta vērtība 1.

P2. var tornis=16;

Šajā rindiņā tiek definēts mainīgais **tornis** un tam piešķirta vērtība 16.

P3. var ziema=vasara;

Šajā rindiņā tiek definēts mainīgais **ziema** un tam piešķirta vērtība, tāda pati kā mainīgajam **vasara**.

P4. var skaitlis="pieci";

Šajā rindiņā tiek definēts mainīgais **skaitlis** un tam piešķirta vērtība "pieci", bet šī vērtība ir teksts, nevis skaitlis.

Atceries – **var** raksta tikai katram mainīgajam vienu reizi, to izveidojot, nedefinējot. Pēc tam kodā tālāk var vienkārši rakstīt **skaitlis="divi"**, jo mainīgais jau ir definēts.

Lapaspuše 3 no 9.

Dažādu programmu izveidē bieži vajag, lai kaut kas tiek izvēlēts uz labu laimi (random) – piemēram, uzprogrammēt metamā kauliņa virtuālo variantu – kad izkrīt jebkurš skaitlis no 1 līdz 6.

var thing = Math.random();

Šī koda rindiņa piešķir mainīgajam **thing** uz labu laimi izvēlētu vērtību starp 0 un 1. Reizēm var tikt piešķirta arī vērtība – 0 (nulle), bet nekad netiks piešķirta vērtība – 1 (viens).

Tātad mainīgais **thing** iegūs vērtību, piemēram, 0.1234 vai 0.0001 vai 0.99643 u.t.t

Piemēri:

P1. var a=Math.random();

Šajā rindiņā tiek definēts mainīgais **a** un tam piešķirta Random vērtība no 0 līdz 1.

P2. ctx.fillRect(100, 100, Math.random(), Math.random());

Šajā rindiņā tiek zīmēts četrstūris, novietojuma koordinātas ir 100 un 100, bet izmērs būs mainīgs katrā koda izpildes reize, jo izmēra skaitļu vietā ir Random vērtības. Tātad izmērs var būt 0.12 un 0.56 vai arī 0.01 un 0.02 u.t.t

Atceries – jebkura skaitļa vietā vari ierakstīt **Math.random()** un šī vērtība kļūst mainīga, izvēlēta uz labu laimi no 0 līdz 1.

Lapaspuše 4 no 9.

Ļoti reti gan mums vajag Random skaitli no 0 līdz 1, jo, ja kaut kas jāzīmē u.c, tur skaitļi pārsvarā būs no 0 līdz pat 1000.

Ja mums vajag, lai iegūtais skaitlis ir apgabalā, piemēram, no 0 līdz 20, tad mēs varam mūsu iegūto Random skaitli pareizināt ar 20.

var thing = Math.random() * 20;

Ja **Math.random()** iedod vērtību tuvu 0, tad pareizinot ar 20, šī vērtība joprojām būs tuvu 0. Ja **Math.random()** iedod vērtību tuvu 1, tad pareizinot ar 20, šī vērtība būs tuvu 20. Tādējādi mēs iegūsim šo Random vērtību no 0 līdz 20.

Piemēri:

P1. var c=Math.random()*10;

Šajā rindiņā tiek definēts mainīgais **c** un tam piešķirta Random vērtība no 0 līdz 10.

P2. `ctx.fillRect(Math.random()*70, 100, 50, Math.random()*40);`

Šajā rindiņā tiek zīmēts četrstūris, novietojuma koordināta X ir Random skaitlis no 0 līdz 70, Y koordināta būs nemainīga – 100, platums ir nemainīgs – 50, bet augstums ir Random skaitlis no 0 līdz 40.

Izpildot šo kodu vairākas reizes, tiks sazīmēti vairāki četrstūri, visi vienā līnijā, jo Y koordināta būs nemainīga. Savukārt četrstūri būs atšķirīgi, ko katram būs atšķirīgs augstums.

Lapaspuse 5 no 9.

Bet mūsu iegūtie skaitļi nav veseli, piemēram, 1.234 vai 5.29346

Tāpēc pielietosim skaitļu apaļošanas iespēju.

Math.floor (skaitlis)

Šī komanda atmet visu, kas ir aiz komata un patur tikai skaitļa veselo daļu.

Ja mēs vēlamies iegūt Random skaitli no 0 līdz 19, tad rakstām šādu kodu:

var thing = Math.floor(Math.random() * 20);

Ar šo rindiņu nekad neiegūsim skaitli 20, jo Math.random nekad nebūs 1.

Lai iegūtu veselu skaitli no 1 līdz 20 pierakstām klāt+1

var thing = (Math.floor(Math.random()*20)) + 1;

Savukārt lai izveidotu metamo kauliņu (skaitļi no 1 līdz 6) mums kods būtu šāds:

var thing = (Math.floor(Math.random()*6)) + 1;

Piemēri:

P1. `var d= 7.158;`

`var a=Math.floor (d);`

Pirmajā rindiņā tiek definēts mainīgais **d** un tam piešķirta vērtība 7.158. Otrajā rindiņā tiek definēts mainīgais **a** un tam tiek piešķirta vērtība, kāda sanāktu, noapaļojot mainīgo **d**. Tātad mainīgā **a** vērtība būs 7.

P2. `var suns = Math.floor(Math.random() * 123);`

Šajā rindiņā tiek definēts mainīgais **suns** un tam piešķirta Random vērtība no 0 līdz 122.

P3. `var car = Math.floor(Math.random() * 18)+1;`

Šajā rindiņā tiek definēts mainīgais **car** un tam piešķirta Random vērtība no 1 līdz 19.

P4. `var cipars = Math.floor(Math.random() * 58)+12;`

Šajā rindiņā tiek definēts mainīgais **cipars** un tam piešķirta Random vērtība no 12 līdz 70.

P5. `ctx.fillRect(Math.floor(Math.random() * 20)+10, Math.floor(Math.random() * 20)+10, 50, 50);`

Šajā rindiņā tiek zīmēts četrstūris, novietojuma koordināta X ir Random skaitlis no 10 līdz 30, novietojuma koordināta Y arī Random skaitlis no 10 līdz 30, platums un augstums ir nemainīgi – 50.

Izpildot šo kodu vairākas reizes, tiks sazīmēti vairāki četrstūri, katrs savā vietā, jo ir mainīgas X un Y koordinātas. Savukārt visi četrstūri būs vienādi, jo platums un augstums ir nemainīgi skaitļi.

Lapaspuse 6 no 9.

`ctx.fillRect(Math.floor(Math.random() * 20), Math.floor(Math.random() * 20), Math.floor(Math.random() * 20), Math.floor(Math.random() * 20),);`

Kā redzat, lai uzzīmētu četrstūri ar 4 Random vērtībām, kods jau kļūst ļoti nepārskatāms. Lai izvairītos no milzīgas kodu kopēšanas un pašu kodu izveidotu pārskatāmu, izveidosim funkciju, kura ģenerēs Random skaitli.

Šeit parādīta funkcija, kura ļauj mums iegūt Random skaitli no 0 līdz funkcijā norādītajai vērtībai.

```
function RandInt(up_to) {  
    n = Math.floor(Math.random() * up_to);  
    return(n);  
}
```

Funkcija piešķir mainīgajam **n** uz labu laimi izvēlētu vērtību no 0 līdz **up_to** (vesels skaitlis).

Izsaukt funkciju varam ar rindiņu:

```
var thing= RandInt(50);
```

Šajā gadījumā mainīgajam **thing** tiek piešķirta vērtība no 0 līdz 49.

Piemēri:

P1. var koks = RandInt(41);

Šajā rindiņā tiek definēts mainīgais **koks** un tam piešķirta Random vērtība no 0 līdz 40. Šī koda rindiņa strādās, ja kodā pirms tam būs izveidota funkcija RandInt.

Ieteikums – vienkārši iekopējiet funkcijas RandInt izveides rindiņas Jūsu kodā un izsauciet pēc vajadzības. Tā būs mazāk iespēju kļūdīties.

P2. var b = RandInt(8)+12;

Šajā rindiņā tiek definēts mainīgais **b** un tam piešķirta Random vērtība no 12 līdz 19. Šī koda rindiņa strādās, ja kodā pirms tam būs izveidota funkcija RandInt. Paskaidrojums – ja RandInt atgriezīs mazāko iespējamo skaitli 0, tad **b** būs 0+12=12, ja RandInt atgriezīs lielāko iespējamo skaitli 7, tad **b** būs 7+12=19

P3. ctx.fillRect(RandInt(101), RandInt(101), RandInt(51), RandInt(51));

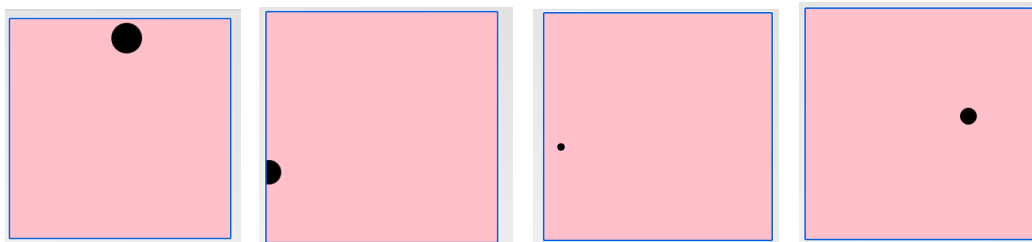
Šajā rindiņā tiek zīmēts četrstūris, novietojuma koordinātas X un Y ir Random skaitlis no 0 līdz 100, platums un augstums ir Random skaitlis no 0 līdz 50. Šī koda rindiņa strādās, ja kodā pirms tam būs izveidota funkcija RandInt.

Izpildot šo kodu vairākas reizes, tiks sazīmēti vairāki četrstūri, katrs savā vietā, jo ir mainīgas X un Y koordinātas. Visi četrstūri būs dažādi, jo platums un augstums ir mainīgi skaitļi.

Lapaspuse 7 no 9.

Šeit parādīts kā uz Canvas laukuma tiek zīmēts aplis ar trīs Random vērtībām – X un Y koordinātas apļa centram un izmērs.

Šādi izskatās mani parauga lodziņi blakus kodam. Un katru reizi viņi ir savādāki, jo katru reizi izkrīt citas koordinātas un izmērs melnajam aplim.



```
fillCircle(RandInt(300), RandInt(300), RandInt(30)+1);
```

Rindiņas skaidrojums – izsaukt funkciju fillCircle un iesniegt tai 3 skaitļus, pirmais (koordināta X) no 0 līdz 299,

otrais (koordināta Y) no 0 līdz 299, trešais (radius) no 1 līdz 30.

Atgādinājums – kā izveidojas šī funkcija un kā to izmantot, palabot, skatīt 8.4 ieskautes materiālos vai mācību

✓ **Control: functions, loops, if-then-else**
sadaļā

📺 Video pamācība:

 **Iekļautais saturs:**
[Skatīt saturu](#) ↗
<https://www.youtube.com/embed/i1haVFZ5Mkk?rel=0>



Lapasuse 8 no 9.

Iepriekšējā lapaspusē mēs lietojām kodu:

```
fillCircle(RandInt(300), RandInt(300), RandInt(30));
```

Ja mums ir **Canvas** ar izmēru 300 uz 300, tad šāds kods būtu pareizs. Bet ko darīt ja mums **Canvas** izmērs ir 400 uz 500? Vai arī ja mēs nezinām sava **Canvas** izmērus? Vai arī – piemēram – mēs uztaisām veselu spēli uz **Canvas** izmēra 300x300, bet beigās saprotam, ka laukums tomēr par mazu. Tad nomainot **Canvas** izmēru, būs jāpārskata viss kods, un visur jāpalabo koda rindiņas, kuras izmanto **Canvas** izmēru.

Mums pastāv iespēja Canvas izmēru kodā nenorādīt, bet katru reizi iegūt esošā laukuma izmērus. Tādējādi kods vienmēr pielāgosies laukuma izmēriem.

```
fillCircle(RandInt(myCanvas.width), RandInt(myCanvas.height), RandInt(30));
```

myCanvas.width – Šis koda fragments mums atgriež mūsu Canvas platuma vērtību kā skaitli.

myCanvas.height – Šis koda fragments mums atgriež mūsu Canvas augstuma vērtību kā skaitli.

Atceries! – myCanvas ir mūsu brīvi izvēlēts **Canvas** nosaukums. Ja veidojot jaunu **Canvas**, tā nosaukums būtu – MansLaukums, tad arī pie platuma iegūšanas būtu jāraksta – MansLaukums.width

Piemēri:

P1. var platums = myCanvas.width;

Šajā rindiņā tiek definēts mainīgais **platums** un tam piešķirta **Canvas** platuma vērtība – skaitlis. Ja ir izveidots **Canvas** ar izmēriem 400x500, tad **platums** vērtība kļūs 400.

P2. var perimetrs = myCanvas.width * 2+ myCanvas.height * 2;

Šajā rindiņā tiek definēts mainīgais **perimetrs**. Tā vērtība tiek iegūta, saskaitot divus **Canvas** platumus un divus **Canvas** augstumus. Ja ir izveidots **Canvas** ar izmēriem 400x500, tad **perimetrs** vērtība kļūs 1800.

P3. ctx.fillRect(0, 0, myCanvas.width/2, myCanvas.height/2);

Šajā rindiņā tiek zīmēts četrstūris, novietojuma koordinātas X un Y ir 0, platums ir puse no **Canvas** platuma un

augstums ir puse no **Canvas** augstuma.

Izpildot šo kodu, tiks uzzīmēts četrstūris pa vienu ceturtdaļu no**canvas**. No 0,0 līdz centram.

📺 Video pamācība:

Iekļautais saturs:
Skatīt saturu ↗
<https://www.youtube.com/embed/hhacITHH6QA?rel=0>



Lapaspuse 9 no 9.

Šeit dots paraugs kodam, kurā redzam ka ir izsaukti vairākas skaitļu ģeneratoru funkcijas.

r = RandInt(3);

Tiek piešķirta mainīgajam **r** vērtība 0, 1 vai 2.**r** tiek izmantots, lai uz labu laimi izvēlētos aplim vienu no 3 iespējamām krāsām (ar IF operatoru).

RandInt(myCanvas.width), RandInt(myCanvas.height)

Vēl divi skaitļu ģeneratori ir izmantoti, lai izvēlētos koordinātas apla centram.

RandInt(30)+1

Arī apla izmēru noteikšanai ir lietots skaitļa ģenerators.

Pamainiet doto kodu, paeksperimentējiet. Piemēram – aplu skaitu nomainiet uz 10 vai 1000. Aplā izmēru 1-30 nomainiet uz 1-10.

Vai varat pievienot vēl vienu krāsu aplu zīmēšanai, lai ir četras krāsas apliem?

function CircleRandColour (x, y, radius) { }

Kodā ir izveidota funkcija **CircleRandColour**. Tas izdarīts, lai pēc tam ciklā varētu šo funkciju izsaukt nepieciešamo skaitu reizi. Piemērā – tās ir 100 reizes. Alternatīva būtu, neveidot funkciju, bet iekopēt funkcijā esošo kodu tieši ciklā.

Mans ieteikums – ja iespējams, ka kods vēlāk būs vēl nepieciešams, veidojiet jaunas funkcijas. Tādējādi izvairīsieties no vienāda koda daudzkārtējas kopēšanas.

```
if (r == 0) {  
    ctx.fillStyle = "red";  
}  
else if (r == 1) {  
    ctx.fillStyle= "green";  
}  
else if (r == 2) {  
    ctx.fillStyle= "blue"; }  

```

Šeit – ja r ir 0, tad krāsa tiek piešķirta Sarkana, ja r ir 1, tad krāsa nomainās uz Zaļu, ja r ir 2, tad krāsa kļūst Zila. Izmantojot šo kā paraugu var izveidot arī 4, 5 vai sešas krāsas.

📺 Video pamācība:



Iekļautais saturs:

[Skatīt saturu ↗](#)

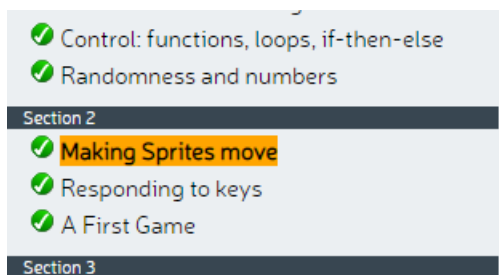
<https://www.youtube.com/embed/uKSm9jqTxzl?rel=0>



2. Tēma - Metode setInterval

Metode setInterval

Turpinām lapas s2js apmācības apskati. Pārejam uz sadaļu – Making Sprites move.



Lapaspuse 1 no 10.

Uz **Canvas** redzam to, ko mēs apgūsim šajā sadaļā. Kā likt attēlam (vai arī jeb kam citam) kustēties.

Lapaspuse 2 no 10.

Atsauce uz Scratch. Tur kustību varēja viegli pielikt.

Lapaspuse 3 no 10.

Līdz šim lai , piemēram, uzzīmētu 100 aplus, mēs lietojām ciklus un visi apli uzreiz tika uzzīmēti, jo programma izpildījās ļoti ātri. Ātrumu ierobežoja tikai datora jauda.

Bet ja mūsu mērķis ir izveidot spēlīti vai kādu lēnu, interesantu animāciju, tad mums vajag lai kāda koda daļa pildās lēnāk un neierobežoti ilgi.

Tam JavaScript valodā ir paredzēti **taimeri**.

setInterval(MyTimerHandler, 500);

Šī koda rindiņā ik pa 500 milisekundēm tiks izpildīta funkcijai **MyTimerHandler**.

1 milisekunde ir 1/1000 daļa no sekundes. Tāpēc 500 milisekundes būs puse no sekundes. Ja mēs gribētu lai funkcija izpildās ik pa sekundi, tad rakstītu 1000.

Piemēri:

P1. setInterval(dari, 1000);

Šajā rindiņā tiek iedarbināts taimeris, kurš izsauks funkciju **dari** 1 reizi sekundē. Tātad jebkas, ko mēs būsim ielikuši šajā funkcijā, tiks izpildīts daudzas reizes. Minūtē 60 reizes, stundā 3600 reizes.

P2. setInterval(DD, 10);

Šajā rindiņā tiek iedarbināts taimeris, kurš izsauks funkciju **DD** katras 10 milisekundes, jeb 100 reizes sekundē. Tātad jebkas, ko mēs būsim ielikuši šajā funkcijā, tiks izpildīts daudzas reizes. Minūtē 6000 reizes, stundā

360000 reizes.

Lapaspuse 4 no 10.

Šeit apskatīts kods, kurā tiek zīmēti kvadrāti.

setInterval(MyTimer, 500);

Katru pussekundi tiek izsaukta funkcija MyTimer.

```
function MyTimer () {  
  y_pos = y_pos + 4;  
  ctx.strokeRect(120, y_pos, 60, 60);  
}
```

Kvadrāta Y koordinātai tiek izmantots mainīgais **y_pos**, kurš sākumā ir 0, bet katru pussekundi izsauktā funkcija pieskaita klāt 4 un tad uzzīmē kvadrātu jau jaunajās koordinātās. Kvadrāta X koordināta nemainās, tā ir fiksēta, tāpēc kvadrāts uz sāniem nekustas.

Ir divi veidi kā paātrināt vai palēnināt kvadrāta kustības ātrumu. Vai nu pamainīt funkcijas izsaukšanas laiku setInterval vai arī izmainīt **y_pos** pieskaitāmā skaitļa lielumu. Izmēģini abus variantus.

Piemēri:

```
P1. var b=0;  
setInterval( riepa, 100);  
function riepa() {  
  b = b + 2;  
  ctx.beginPath();  
  ctx.arc(150, 150, b, 0, Math.PI * 2);  
  ctx.fill();  
}
```

Šajā rindiņā tiek iedarbināts taimeris, kurš izsauks funkciju **rieпа** 10 reizes sekundē. Funkcija zīmē aplus. Apļa centrs nemainās un ir 50,150. Savukārt rādiuss **b** katrā izpildes reizē tiek palielināts par 2. Tātad tiks uzzīmēti daudz apli, kur katrs nākamais būs lielāks par iepriekšējo.

Lapaspuse 5 no 10.

Kā redzējāt iepriekšējā lapaspusē – mēs sazīmējām daudz kvadrātu. Bet mums gribētos, lai uz **Canvas** ir tikai 1 kvadrāts, kurš katru reizi ir citā vietā – pārvietojas.

Tagad mums ir 2 varianti –

1. – mēs varētu mēģināt zīmēt pa virsu iepriekšējam taisnstūrim ar kaut **kocanvas** krāsā, tādējādi noslēpjot šo taisnstūri.

2. – notīrīt Canvas pilnīgi tīru katru reizi, pirms zīmēt jauno kvadrātu.

Otrajā variantā mēs varam lietot šādu koda rindiņu:

ctx.clearRect(0, 0, myCanvas.width, myCanvas.height);

ctx.clearRect notīra norādīto laukumu **Canvas**. Piemērā dotajā koda rindiņā tiek nodzēsts pilnīgi viss **Canvas**, jo sākuma koordinātas ir 0,0 un beigu koordinātas ir viss **Canvas** platums un augstums.

Šī koda rindiņa mūsu piemērā ir jāiekļauj funkcijā, jo tai jāizpildās katru reizi pirms jaunā kvadrāta uzzīmēšanās, bet Jūs viņu varat lietot arī citos gadījumos, arī ārpus funkcijas, kad vien vajag, lai **Canvas** būtu tīrs.

Piemēri:

```
P1. ctx.strokeRect(100, 100, 50, 50);
ctx.clearRect(0, 0, myCanvas.width, myCanvas.height);
ctx.strokeRect(100, 100, 70, 70);
ctx.clearRect(0, 0, myCanvas.width, myCanvas.height);
ctx.strokeRect(100, 100, 90, 90);
```

Šāds kods nav loģisks, bet apskatīsim. Tiek uzzīmēts taisnstūris ar izmēru 50,50. Tad viss **Canvas** tiek notīrīts. Tiek uzzīmēts taisnstūris ar izmēru 70,70. Tad viss **Canvas** tiek notīrīts. Tiek uzzīmēts taisnstūris ar izmēru 90,90.

Gala rezultātā mums būs tikai viens taisnstūris, jo pilnīgi viss, kas bija uz **Canvas** pirms tam tika notīrīts.

```
P2. if a>10 {
ctx.clearRect(0, 0, myCanvas.width, myCanvas.height); }
```

Šeit viss **Canvas** tiks notīrīts, ja mainīgā **a** vērtība būs lielāka par 10. Pārējos gadījumos **Canvas** netiks notīrīts.

```
P3. setInterval( zils, 100);
function zils() {
  b = b + 2;
  ctx.beginPath();
  ctx.arc(150, 150, b, 0, Math.PI * 2);
  ctx.fill();
ctx.clearRect(0, 0, myCanvas.width, myCanvas.height); //Nodzēšanas rindiņa
}
```

Šeit parādīts kļūdainais kods. Lai arī funkcija **zils** tiek izsaukta 10 reizes sekundē un tā uzzīmē apli, apli tomēr mēs neredzēsim, jo katrā funkcijas izpildes reizē šis aplis tiek nodzēsts.

Nodzēšanas rindiņai būtu jābūt funkcijas sākumā, vispirms notīrīt un tad uzzīmēt.

📺 Video pamācība:

 **Iekļautais saturs:**
Skatīt saturu ↗
<https://www.youtube.com/embed/NZMDwIVH89U?rel=0>



Laspuse 6 no 10.

Ja gribas, lai kvadrātu kustība būtu plūdenāka, ne tik saraustīta, tad jāpalabo šī rindiņa.

```
setInterval(MyTimerHandler, 40);
```

Vēlams uzlikt 40, kas dotu standarta 1000/40 = 25 pārzīmēšanas sekundē. Tas ir diezgan izplatīts skaitlis, kurš tiek izmantots daudzās spēlēs, video u.c

Bet ja mēs vienkārši palielināsim izpildes ātrumu, tad kvadrāts skries pa ātru, tāpēc jāsamazina pārvietošanās solis. Tātad labāk daudz mazu soliņu, nevis daži lieli.

Lapaspuse 7 no 10.

Šeit piemērs ar diviem krītošiem taisnstūriem.

Vai varat uzrakstīt kodu, lai taisnstūru vietā kristu aplīši?

Vai varat uzrakstīt kodu, kurā šie taisnstūri nevis krīt, bet ceļas uz augšu no pašas apakšas?

```
function MyTimer () {  
  ctx.clearRect(0, 0, myCanvas.width, myCanvas.height);  
  y_pos = y_pos + 1;  
  ctx.strokeRect(100, y_pos, 60, 60);  
  ctx.fillRect(100, y_pos + 80, 60, 60);  
}
```

Pamēģini palabot kodu mācību sadaļā, lai tas izskatās šādi. Tagad abi četrstūri ir viens zem otra.

Lapaspuse 8 no 10.

Ja mums pārvietosies tikai kvadrāti vai aplī, nebūs tik smuki, kā ja mums pārvietosies kāds attēls.

Lai ievietotu attēlu **Canvas**, lietojam šādas koda rindiņas:

```
var MyImg = new Image();  
MyImg.src = "https://s2js.com/img/etc/cat_grumpy.png";
```

Šīs rindiņas jāraksta vienreiz, koda sākumā, kur definējam mainīgos. Un tālāk jau kodā atsauksimies uz šīm ienestajām bildēm. Kā vienmēr – mainīgajam **MyImg** vietā rakstīt jebkuru Jūsu izvēlētu nosaukumu.

Šīs rindiņas nezīmē attēlu, tikai to definē, piesaista konkrētam mainīgajam.

Savukārt lai šo ienesto attēlu ievietotu mūsu Canvas vajag sekojošu koda rindiņu:

```
ctx.drawImage(MyImg, 100, 200);
```

MyImg – mainīgais, kurš definēts kā attēls.

100 – Attēla X koordināta

200 – Attēla Y koordināta

Ja šī rindiņa būs iekšā funkcijā, ko izsauc setInterval, tad problēmu nebūs, attēls tiks uzzīmēts.

Bet ir viena problēma ar JavaScript – ja vienkārši kodā iekopē attēla zīmēšanas rindiņu, tad parasti tas netiek uzzīmēts. Tāpēc kodā jālieto šāda rindiņa:

```
MyImg.onload = function() {ctx.drawImage(MyImg, 100, 100);}
```

Ar šo rindiņu attēls tiks uzzīmēts, lai arī kurā vietā kodā šī rindiņa būtu.

Piemēri:

```
P1. var Bilde1 = new Image();  
Bilde1.src = "https://s2js.com/img/etc/cat_grumpy.png";  
var Bilde2 = new Image();  
Bilde2.src = "https://s2js.com/img/etc/ladybug.png";
```

```
MyImg.onload = function() {ctx.drawImage(Bilde1, 100, 100);}
```

```
MyImg.onload = function() {ctx.drawImage(Bilde2, 200, 200);}
```

Ja mums uz **Canvas** vajag vairākus attēlus, tad attiecīgi vajag arī vairāk mainīgos. Katram mainīgajam var piesaistīt vienu attēlu.

```
P2. var Zirgs = new Image();  
Zirgs.src = "https://s2js.com/img/etc/cat_grumpy.png";  
MyImg.onload = function() {ctx.drawImage(Zirgs, 50, 70);}
```

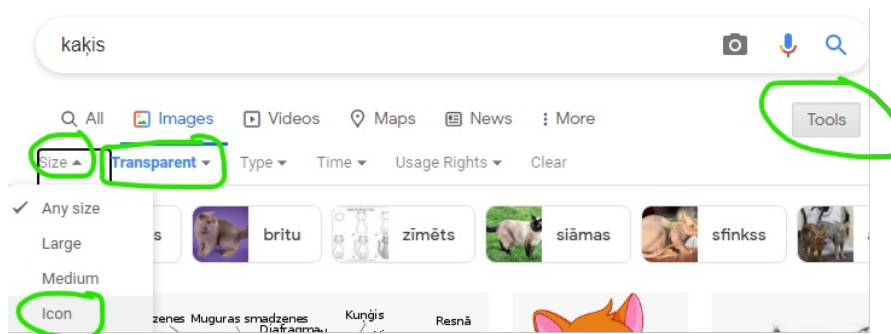
Tātad, jābūt rindiņai kas nodefinē mainīgo kā attēlu, tad rindiņa, kura pasaka šim mainīgajam, kur tieši ir šis attēls un koda rindiņa, kura uzzīmē šo attēlu.

Lapaspuse 9 no 10.

Atkal vietīņa paspēlēties ar kodu. Pamēģini ienest kādu savu attēlu, sameklētu Google.

Tā kā mūsu **Canvas** parasti nav liels, tad, visticamāk, mums vajadzēs mazu attēlu. Lai Google sameklētu mazus attēlus, Izvēlaties Tools un tad pie Size izvēlaties – Icons.

Savukārt pie Color izvēlaties – Transparent. Tādējādi iegūsim attēlu ar caurspīdīgu fonu, un tas izskatīsies labāk.



Un tad atliek attēlu atvērt un nokopēt tā saiti. Un tad iekopēt to kodā.

📺Video pamācība:

**Iekļautais saturs:**
[Skatīt saturu](#) ↗
<https://www.youtube.com/embed/4nZKjrcF35k?rel=0>



Lapaspuse 10 no 10.

Aplūko kodu un mēģini saprast ko katra rindiņa dara. Kura rindiņa liek kaķim krist slīpi? Kura rindiņa liek kaķim celties uz augšu?

Ja nesanāk, tad izskaidrošu.

```
function MyTimer () {  
  ctx.clearRect(0, 0, myCanvas.width, myCanvas.height);  
  y_pos = y_pos + 1;
```

```
ctx.drawImage(MyImg, 130, y_pos);  
ctx.drawImage(MyImg, 50, myCanvas.height - y_pos);  
ctx.drawImage(MyImg, y_pos, y_pos);  
}
```

Kodā ir tikai 1 mainīgais kā attēls, jo visi attēli ir vienādi.

Funkcijā 1. rindiņa katrā cikla izpildes reizē notīra visu **Canvas**.

Otrā rindiņa katrā cikla izpildes reizē palielina mainīgā **y_pos** vērtību.

Trešā rindiņa uzzīmē attēlu, kur X koordināta ir nemainīga, bet Y koordināta pieaug, tātad attēls virzās uz leju.

Ceturtā rindiņa uzzīmē attēlu, kur X koordināta ir nemainīga, bet Y koordināta tiek samazināta. Sākotnēji Y pozīcija ir pašā **Canvas** apakšā, bet tā kā **y_pos** pieaug, tad atņemot no **Canvas** augstuma **y_pos** vērtību, gala skaitlis paliek ar vien mazāks. Attēls virzās uz augšu.

Piektā rindiņa uzzīmē attēlu, kur gan X gan Y pozīcija ir vienāda, bet katrā cikla izpildes reizē pieaug. Tātad attēls virzās pa diagonāli.

Piemēri:

P1. ctx.drawImage(MyImg, y_pos, 100);

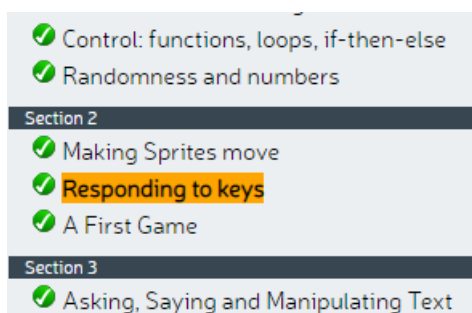
Šajā rindiņā attēls virzītos pa labi, jo Y koordināta ir nemainīga, bet X koordināta katrā cikla izpildes reizē pieaugtu.

3. Tēma - Klaviatūras pogu izmantošana

Klaviatūras pogu izmantošana

Dažas jaunas iespējas JavaScript esam apguvuši. Turpinām ar nodaļu, kurā uzzināsim kā klaviatūras pogu nospiešanai piešķirt darbības Javascript vidē.

Pārejam uz sadaļu – Responding to keys



Lapasuse 1 no 13.

Pamēģini nospiegt bultiņas klaviatūrā – bumba pārvietojas uz to pusi, kur liekam tai virzīties. Ja mēs mēģināsim izveidot kādu spēli, tad ļoti svarīgi ir spēt vadīt objektus. Iesim tālāk un apskatīsim to izdarīt.

Lapasuse 2 no 13.

Atkal salīdzinājums ar Scratch. Arī JavaScript mums būs jāizmanto jēdziens “Notikumi” - **Events**

Lapasuse 3 no 13.

JavaScript valodā ir jēdziens – **Events**. Notikumi. Tie ir signāli, kuri tiek radīti pie konkrētiem notikumiem. Piemēram, peles klikšķis, pogas nospiešana, mājas lapas ielādes beigšanās u.c JavaScript šos signālus var uztvert un mēs varam izveidot kodu, kurš izpildās, ja kāds no šiem signāliem ir uztverts.

`addEventListener("keydown", MyKeyDownHandler);`

Šī koda rindiņa norāda, ka mēs gaidām, kad tiks nospiesta kāda klaviatūras poga - **"keydown"** un tad tiek izsaukta funkcija **MyKeyDownHandler**. Funkcijas nosaukums brīvi izvēlēts.

Katram Event ir parametri, kuri nāk līdzi. Piemēram, ja nospiežam klaviatūras pogu, tad tiek pateikts arī kura poga bija nospiesta. Ja Event ir peles pakustināšana, tad parametros būs jaunās koordinātas u.c

MyEvent.keyCode – šis koda fragments mums atgriezīs informāciju par to, kura tieši poga ir nospiesta. Tas būs skaitlis.

```
function MyKeyDownHandler (MyEvent) {
    // Darīt kaut ko ar MyEvent.keyCode
}
```

Tālāk jau izsauktajā funkcijā varam izmantot šo informāciju pēc vajadzības.

Piemēri:

```
P1. addEventListener("keydown", Sist);
function Sist (MyEvent) {
    var a= MyEvent.keyCode;
}
```

Šajā kodā tiek pateikts ka tiek gaidīta klaviatūras pogas nospiešana un ja tā notiek, tiek izsaukta funkcija **Sist**. Savukārt izsauktā funkcija **Sist** mainīgajam **a** piešķir nospiebtā klaviatūras taustiņa koda vērtību.

Lapaspuše 4 no 13.

Šeit izveidojam pamatu mūsu attēla pārvietošanas kodam.

Līdz šim mums bija tikai mainīgais **y_pos**, jo pārvietojām attēlu vienā virzienā. Tagad mums vajag nodefinēt divus mainīgos **X** un **Y** koordinātām.

Piemērā tie ir:

```
var x_pos = 0;
var y_pos = 0;
```

Kā arī izveidot koda rindiņas attēla ienešanai.

Ar **setInterval** palīdzību mums jānodefinē arī tas, ka attēls tiek zīmēts dotajās koordinātās un pārzīmēts 25 reizes sekundē. Šo jau bijām darījuši. Atšķirība ir tikai:

```
ctx.drawImage(MyImg, x_pos, y_pos);
```

Ir izmantots viens mainīgais **X** koordinātai un cits **Y** koordinātai. Tādējādi mainot šīs divas vērtības, attēls var tikt uzzīmēts jebkurā vietā

Rezultātu jūs redzat – attēls ar kaķi tiek uzzīmēts stūrītī un nepārvietojas. Jo pašlaik vēl nav nekas, kas mainītu attēla **X** un **Y** koordinātas. Tās ir un paliek 0,0.

Lapaspuše 5 no 13.

Pievienosim kodam **Event**, kurš aktivizēs funkciju pēc pogas nospiešanas un pašu funkciju:

```
addEventListener("keydown", MyKeyDownHandler);
```

```
function MyKeyDownHandler (MyEvent) {
    if (MyEvent.keyCode == 39) {x_pos = x_pos + 10;}
    if (MyEvent.keyCode == 40) {y_pos = y_pos + 10;}
}
```

Tā kā uz klaviatūras ir daudz pogu, tad katrai pogai ir savs apzīmējuma cipars. Šajās koda rindīnās redzam, ka **39** – tā ir poga “pa labi” un **40** – poga “uz leju”.

Lai katrai konkrētai pogai piešķirtu darbību, mums jālieto **IF** (to apskatījām 8. klasē).

Kods lasās šādi:

ja **MyEvent.keyCode == 39**, jeb nospiesta poga “**pa labi**”, tad izpildām kodu **{x_pos = x_pos + 10;}**. Tādējādi attēla **X** koordināta tiek palielināta par 10. Un attēls pārvietojas pa labi.

ja **MyEvent.keyCode == 40**, jeb nospiesta poga “**uz leju**”, tad izpildām kodu **{y_pos = y_pos + 10;}**. Tādējādi attēla **Y** koordināta tiek palielināta par 10. Un attēls pārvietojas uz leju.

Uz citām nospiestām pogām reaģēts netiks.

Un tā kā mums ir arī funkcija taimerim, kura visu laiku pārzīmē attēlu, tad nomainot tā koordinātu vērtības, pārvietojas arī pats attēls.

!!! Ņem vērā – lai pogas strādātu un kaķis pārvietotos, **Canvas** jābūt aktīvam. Ja kaķis nepārvietojas, uzklikšķini ar peli uz **Canvas**.

Virziena pogu kodi:

37 – poga “pa kreisi”

38 – poga “uz augšu”

39 – poga “pa labi”

40 – poga “uz leju”

Pamēģini pievienot kaķa attēlam visas 4 virzienu pogu darbības.

Piemēri:

```
P1. addEventListener("keydown", Siena);  
function Siena (MyEvent) {  
if (MyEvent.keyCode == 7) { ctx.strokeRect(10, 10, 50, 50);}  
if (MyEvent.keyCode == 8) { ctx.fillRect(20, 20, 60, 60);}  
if (MyEvent.keyCode == 9) { ctx.fillText("Hello", 10, 50);}  
}
```

Šajā kodā tiek pateikts ka tiek gaidīta klaviatūras pogas nospiešana un, ja tā notiek, tiek izsaukta funkcija **Siena**.

Savukārt izsauktā funkcija skatās kura poga ir nospiesta. Ja tiek nospiesta poga ar kodu “7”, tad tiek uzzīmēts neaizpildīts četrstūris. Ja tiek nospiesta poga ar kodu “8”, tad tiek uzzīmēts aizpildīts četrstūris. Ja tiek nospiesta poga ar kodu “9”, tad uz **Canvas** tiek uzrakstīts teksts “Hello”.

Lapaspuse 6 no 13.

Šeit redzam dažu taustiņu kodus.

37 – Bulta pa kreisi
39 – Bulta pa labi
38 – Bulta uz augšu
40 – Bulta uz leju
32 – Atstarpes taustiņš
27 – Atsoļa taustiņš
65-90 – Burti A-Z
48-57 – Skaitļi 0-9
16 - Pārslēgšanas taustiņš
17 – Vadības taustiņš
18 - Alternēšanas taustiņš
13 – Ievades taustiņš
20 - Burtoslēga taustiņš
8 - Atkāpšanās taustiņš
9 - Tabulators

Key	keyCode
Left arrow	37
Right arrow	39
Up arrow	38
Down arrow	40
Space bar	32
Escape	27
A - Z	65-90
0 - 9	48-57
Shift	16
Control	17
Alt	18
Enter	13
CapsLock	20
Backspace	8
Tab	9

Sarunvalodā bieži nelietojam latviskos taustiņu nosaukumus, bet būtu tos tomēr labi zināt, tāpēc ieskaitē noteikti par to būs jautājums.

Lapaspuse 7 no 13.

Varbūt jau pamanīji – Javascript neierobežo Tevi tikai **Canvas** laukuma robežās, Attēlu mūsu programmā var izbīdīt ārpus **Canvas** un vairs neredzēt. Bet tas, ka mēs to neredzam, nemaina neko paša attēla kustībā – viņš turpina bīdīties norādītajā virzienā, ja spiežam virzienbultas.

Šajā koda piemērā ir pievienoti arī pārējie divi kustību virzieni. Tagad attēlu varam kustināt visos četros virzienos.

Lapaspuse 8 no 13.

Kā Tev šķiet, ko vajadzētu darīt, lai attēls neizietu ārpus laukuma?

Ja Tev ir kāda izdeja, tad pamēģini uzrakstīt kodu, kas ierobežo attēla kustību pa kreisi tālāk pa laukuma robežu. Nākamajās lapaspusēs varēsi pārbaudīt vai ir izdomāts pareizi.

Te gan jāsaprot ka JavaScript var būt desmitiem veidu kā atrisināt vienu un to pašu problēmu. Un visi būs pareizi, ja gala rezultāts būs panākts.

Mājas darbos un ieskaitē gan šo brīvību es ierobežošu, jo drīkstēs izmantot tikai kodu, ko esam apguvuši līdz šim. Jebkurš kods no citām mājas lapām tiks uzskatīts par nepareizu.

Lapaspuse 9 no 13.

Šeit ir viens no risinājumiem, kā ierobežot attēla kustību pa kreisi un uz augšu.

```
if (MyEvent.keyCode == 37 && x_pos > 0) {x_pos = x_pos - 10;}
```

Papildinām koda rindiņu, ka mainīgā **x_pos** vērtība tiek samazināta par desmit tikai tad, ja tā ir lielāka par 0. Un tai brīdī, kad attēls tiks piebīdīts pie pašas malas un X koordināta būs 0, attēls tālāk nebīdīsies.

&& tulkojas kā **AND (UN)**. Tas nozīmē, ka IF funkcija tiks izpildīta tikai tad, ja abi tās nosacījumi būs patiesi. Nepietiks, ja izpildīsies tikai viens no abiem nosacījumiem.

Pamēģini pabīdīt kaķi pa kreisi vai uz augšu – to vairs nevar izbīdīt ārpus laukuma.

Bet ko lai dara ar attēla pārvietojuma ierobežošanu uz leju un pa labi?

Lapaspuse 10 no 13.

Viens variants – uzrakstīt šādu koda rindiņu:

```
if (MyEvent.keyCode == 39 && x_pos < 300) {x_pos = x_pos + 10;}
```

Šāda koda rindiņa strādās, ja **Canvas** izmērs ir 300. Bet vēl labāk būtu uzrakstīt šādi:

```
if (MyEvent.keyCode == 39 && x_pos < myCanvas.width)
{x_pos = x_pos + 10;}
```

Mēs jau iemācījāmies ka **myCanvas.width** nolasa mūsu **Canvas** platumu un mums nav jāzin cik tieši plats tas ir.

Bet. Pabīdi kaķi līdz apakšai un paskaties vai šāds kods strādās. Vai kaķis palika laukuma robežās, vai arī izbīdījās ārpus tā? Kaķis tomēr pazuda, izgāja ārpus **Canvas**. Kas notika, kāpēc radās problēma?

Lapasuse 11 no 13.

Šeit ir atbilde problēmai. Attēla koordinātas ir augšējais kreisais stūris. Un augšējais kreisais stūris arī neiziet ārpus **Canvas** robežām. Bet pārējā bilde iziet ārpus **Canvas**, jo netiek ņemts vērā pašas bildes platums un augstums.

Pareizi būtu, ja mēs bildes labo stūri izmantotu kontrolei - ir attēls izgājis ārpus **canvas** vai nav. Ja bildes kreisā stūra x koordināta mums bija **x_pos**, tad labā stūra pozīcija būtu

x_pos + MyImg.width

X koordinātas un bildes platuma summa. Kā redzat, tad platuma noteikšana strādā ne tikai **Canvas**, bet arī attēliem - **.width**

Lapasuse 12 no 13.

Šeit mums tiek parādīts kāds būtu pareizais risinājums attēla ierobežošanai Canvas laukumā.

```
function MyKeyDownHandler (MyEvent) {  
  if (MyEvent.keyCode == 37 && x_pos > 0) {x_pos = x_pos - 10;} // left  
  if (MyEvent.keyCode == 38 && y_pos > 0) {y_pos = y_pos - 10;} // up  
  if (MyEvent.keyCode == 39 && x_pos+MyImg.width < myCanvas.width) {x_pos = x_pos + 10;} // right  
  if (MyEvent.keyCode == 40 && y_pos+MyImg.height < myCanvas.height) {y_pos = y_pos + 10;} // down  
}
```

1. rindiņa – tikai nosacījums **x_pos > 0**. Ja bīdām attēlu pa kreisi, tad varam pārbaudīt tikai vai kreisais stūris nav sasniedzis kreiso malu.

2. rindiņa – tikai nosacījums **y_pos > 0**. Ja bīdām attēlu uz augšu, tad varam pārbaudīt tikai vai kreisais stūris nav sasniedzis augšējo malu.

3. rindiņa – nosacījums **x_pos+MyImg.width < myCanvas.width**. Ja bīdām attēlu pa labi, tad jāpārbauda vai attēla labais stūris nav sasniedzis **canvas** labo malu. Attēla X koordinātai pieskaitām attēla platumu un ja šis iegūtais skaitlis (attēla labā stūra novietojuma X koordināta) ir mazāks par Canvas platumu, tātad vēl varam bīdīt attēlu un tad **x_pos** palielinām par 10.

4. rindiņa – nosacījums **y_pos+MyImg.height < myCanvas.height**. Ja bīdām attēlu uz leju, tad jāpārbauda vai attēla apakšējais stūris nav sasniedzis **canvas** apakšu. Attēla Y koordinātai pieskaitām attēla augstumu un ja šis iegūtais skaitlis (attēla apakšējā stūra novietojuma Y koordināta) ir mazāks par Canvas augstumu, tātad vēl varam bīdīt attēlu un tad **y_pos** palielinām par 10.

Pabīdiet kaķi, un paskatieties – viss strādā vai tomēr nē? Pa lielam viss notiek, bet neliels attēla gabaliņš var iziet ārpus laukuma, jo mūsu bīdīšanās solis ir +10. Bet ja no attēla stūra līdz Canvas malai palicis vairs tikai 5? Nosacījums būs patiens un attēls drusku pārlēks pāri malai.

Šoreiz gan to noignorēsim, jo mums tik lielu precizitāti nevajag.

Lapasuse 13 no 13.

Bīdot kaķi mums bija viena traucējoša problēma. Nospiežot virzienbultu, mēdz bīdīties ne tikai attēls, bet visa mājas lapa. Tas ir tāpēc, ka arī interneta pārlūks gaida pogu nospiešanu un reaģē uz tām.

Bet mums ir iespēja pārlūkprogrammai pateikt, lai ignorē mūsu klaviatūras pogu nospiešanu.

Jāpievieno šī papildus rindiņa kodā.

MyEvent.preventDefault();

Rindiņa - **MyEvent.preventDefault();** - ievietota pogas izmantošanas funkcijā, pateiks pārlūkprogrammai, ka tai

šī pogas nospiešana ir jāignorē.

📺 Video pamācība:



Iekļautais saturs:

Skatīt saturu ↗

<https://www.youtube.com/embed/bTctbUIZTI8?rel=0>



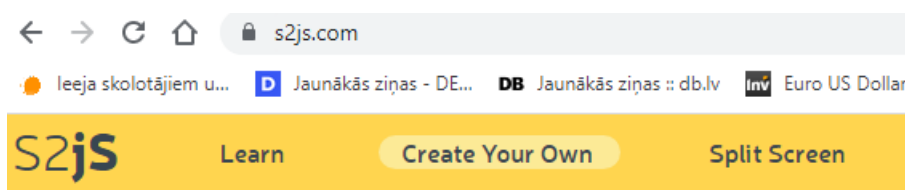
Mājas darbs Nr.1

Mājas darbs

Šajā mācību materiālā apguvām JavaScript valodā Random skaitļu ģeneratoru, Taimera izmantošanu un klaviatūras iesaisti. Nākamajās ieskaitēs turpināsim Javascript valodas apguvi.

Tā pat, kā mēs jau 8. klasē darījām, mājas darba kodu rakstīsim mājas lapā <https://s2js.com/>

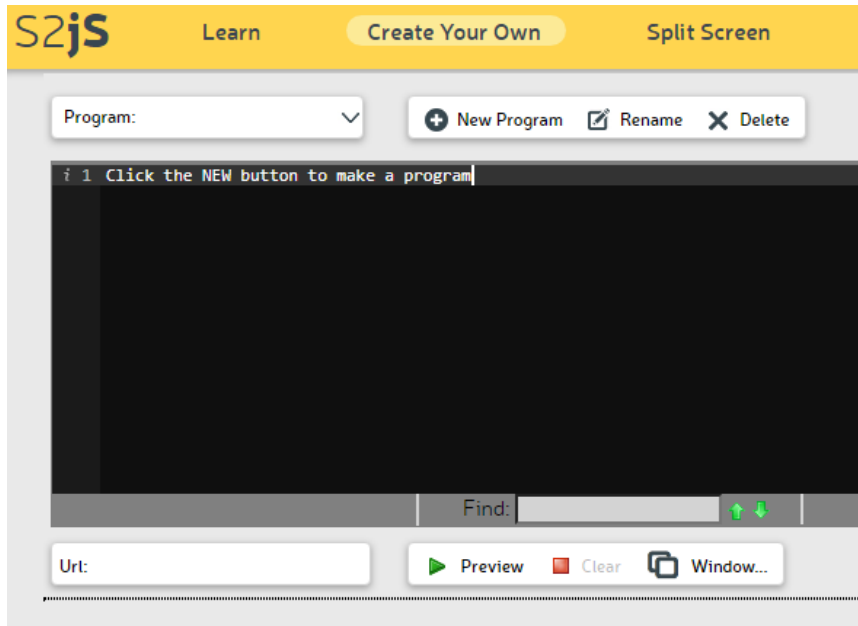
Mēs pāriesim uz šīs mājas lapas sadaļu, kurā Tu varēsi rakstīt savu kodu un to notestēt.



Izvēlies - **Create Your Own**.

Ja vēl neesi pierēģistrējies šajā mājas lapā, tad tagad tas ir jāizdara.

Pēc reģistrācijas Tev atvērsies šāds skats:



Šeit nospied **+ New Program**. Un ieraksti savas programmas nosaukumu.

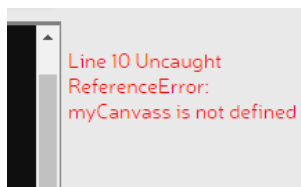
```

1 <HTML>
2   <head>
3     <title> Tests </title>
4   </head>
5
6   <body>
7     <canvas id=myCanvas width=300 height=300 style="background-color: pink;"> </canvas>
8
9     <script>
10      var ctx = myCanvas.getContext("2d");
11
12      // Your code goes here!
13      // If you're not sure how to code Javascript, go back and read the S2JS tutorials
14
15    </script>
16

```

Lodziņā parādīsies tukša sagatave darbam. Audekls jau ir izveidots. Tu vari to palabot, ja vajag. Script tagi arī ir. Starp šiem tagiem Tu vari rakstīt savus JavaScript kodus.

Uzrakstot kodu, var nospiegt pogu  Preview. Un apakšā parādīsies Tevis izveidotās mājas lapas priekšapskats. Ja kods nebūs pareizs, tad labajā pusē parādīsies kļūdas paziņojums. Te tu arī redzēsi kurā rindiņā ir kļūda.



Iekopē šeit kādu koda fragmentu no apmācību sadaļas, paskaties kā strādā šī sadaļa. Atceries, Tev jākopē tikai kods starp Script tagiem, jo pārējais jau sagatavē ir izveidots.

Mājas darbs Nr.1

Nosacījumi:

1. Programmu izveidē drīkst būt tikai kodi, ko esam apskatījuši šajās nodaļās. Jebkurš cits kods tiks uzskatīts par kļūdu un **tiks samazināts punktu skaits**. Darba izveidei pilnībā pietiek ar kodiem, ko esam apskatījuši, papildus nekas nav vajadzīgs. (Piemēram, nelietot masīvus)
2. ChatGPT Tu vari lietot, lai palīdzētu apgūt JavaScript. Bet mājas darba izveidē ChatGPT lietot **nedrīkst**. Ja mūsu sistēma uzrādīs ka darbu vai tā daļu veicis ChatGPT, **darbs var tikt novērtēts ar 0 vai arī tiks samazināts punktu skaits**.

Izveidojiet jaunu programmu mājas lapā <https://s2js.com/>

Mājas darbam ir 3 varianti, kurš ir Tavs – atkarīgs no **Uzvārda** pirmā burtā.

Ieteikums: Ja sāc pildīt mājas darbu un kautkas neizdodas, raksti datorikas skolotājam. Apraksti kas nesanāk un pievieno jau izdarītā darba saiti (linku)

1. variants

1. var. - Ja Tev **uzvārds** sākas ar burtiem **A, Ā, B, C, Č, D, E, Ē, F, G, Ģ**, tad izveidojiet **Canvas**(audeklu) – krāsa - attēlā, ar izmēru 414x414 (2p)

Uz šī Canvas, izmantojot **Random skaitļu ģeneratoru** jāuzzīmē sekojošas figūras
(**Nemt vērā – A ir Jūsu dzimšanas dienas datums, piemēram, man tas ir 25**):

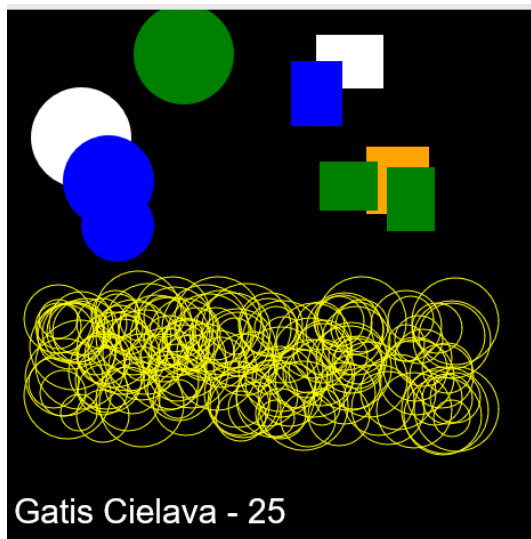
1.(6p) Atšķirīgi četri aizpildīti apli – Izmērs no **A līdz 50**, novietojuma centra koordinātas X un Y no **0 līdz 200**, katram aplim krāsa mainīga - gadījuma izvēle (random) no trim krāsām, krāsas skatīt parauga attēlā. (Koda paraugu vari paņemt no mācību sadaļas)

2.(7p) Pieci aizpildīti taisnstūri – Garums un platums no **20 līdz 60**, novietojuma koordinātas X no **200 līdz 340** un Y no **0 līdz 150**, katram taisnstūrim krāsa mainīga - gadījuma izvēle (random) no četrām krāsām - trīs krāsas kā apliem plus viena papildus. (Izmanto to pašu kodu ko krāsainiem apliem, tikai nomaini figūru un pieliec papildus krāsu)

3. (7p) Izmantojot ciklu, jāuzzīmē **110** neaizpildīti apli dzeltenā krāsā, – Izmērs no **A līdz 40**, novietojuma centra koordinātas X no **0 līdz 380** un Y no **220 līdz 320**

4.(3p) Jāiekļauj arī kods, kurš Canvas apakšā uzraksta Jūsu **vārdu, uzvārdu un kodā izmantoto Jūsu dzimšanas datumu** – Teksta krāsa – **balta**
Teksta izmērs – **30px**

Gala rezultāta aptuvenais izskats (tam jābūt mainīgam katrā koda izpildes reizē):



2. variants

2. var. - Ja Tev **uzvārds** sākas ar burtiem **H,I,Ī,J,K,Ķ,L,Ļ,M,N,Ņ**

tad izveidojiet **Canvas** (audeklu) – krāsa - attēlā, ar izmēru 380x380 (2p)

Uz šī Canvas, **izmantojot Random skaitļu ģeneratoru** jāuzzīmē sekojošas figūras (**Nemt vērā – A ir Jūsu dzimšanas dienas datums, piemēram, man tas ir 25**):

1.(6p) Atšķirīgi četri aizpildīti apli – Izmērs no **A līdz 40**, novietojuma centra koordinātas X un Y no **0 līdz 180**, katram aplim krāsa mainīga - gadījuma izvēle (random) no trim krāsām, krāsas skatīt parauga attēlā. (Koda paraugu vari paņemt no mācību sadaļas)

2.(7p) Pieci aizpildīti taisnstūri – Garums un platums no **15 līdz 50**,
novietojuma koordinātas X no **210 līdz 330** un Y no **0 līdz 140**,
katram taisnstūrim krāsa mainīga - gadījuma izvēle (random) no četrām krāsām - trīs krāsas kā apliem plus viena papildus.
(Izmanto to pašu kodu ko krāsainiem apliem, tikai nomaini figūru un pieliec papildus krāsu)

3. (7p) Izmantojot ciklu, jāuzzīmē **120** neaizpildīti apli violetā krāsā, – Izmērs no **A līdz 45**,
novietojuma centra koordinātas X no **0 līdz 360** un Y no **210 līdz 310**

4.(3p) Jāiekļauj arī kods, kurš Canvas apakšā uzraksta Jūsu **vārdu, uzvārdu un kodā izmantoto Jūsu dzimšanas datumu** –
Teksta krāsa – **zaļa**
Teksta izmērs – **25px**

Gala rezultāta aptuvenais izskats (tam jābūt mainīgam katrā koda izpildes reizē):



3. variants

3. var. - Ja Tev **uzvārds sākas ar burtiem O,P,R,S,Š,T,U,Ū,V,Z,Ž**
tad izveidojiet **Canvas** (audeklu) – krāsa - attēlā, ar izmēru 430x430 (2p)

Uz šī Canvas, **izmantojot Random skaitļu ģeneratoru** jāuzzīmē sekojošas figūras
(Nemt vērā – A ir Jūsu dzimšanas dienas datums, piemēram, man tas ir 25):

1.(6p) Atšķirīgi četri aizpildīti apli – Izmērs no **A līdz 45**, novietojuma centra koordinātas X un Y no **0 līdz 210**,
katram aplim krāsa mainīga - gadījuma izvēle (random) no trim krāsām,
krāsas skatīt parauga attēlā. (Koda paraugu vari paņemt no mācību sadaļas)

2.(7p) Pieci aizpildīti taisnstūri – Garums un platums no **25 līdz 70**,
novietojuma koordinātas X no **220 līdz 370** un Y no **0 līdz 170**,
katram taisnstūrim krāsa mainīga - gadījuma izvēle (random) no četrām krāsām - trīs krāsas kā apliem plus viena papildus.
(Izmanto to pašu kodu ko krāsainiem apliem, tikai nomaini figūru un pieliec papildus krāsu)

3. (7p) Izmantojot ciklu, jāuzzīmē **130** neaizpildīti apli sarkanā krāsā, – Izmērs no **A līdz 50**,
novietojuma centra koordinātas X no **0 līdz 400** un Y no **240 līdz 360**

4.(3p) Jāiekļauj arī kods, kurš Canvas apakšā uzraksta Jūsu **vārdu, uzvārdu un kodā izmantoto Jūsu dzimšanas datumu** –
Teksta krāsa – **sudraba (pelēka)**
Teksta izmērs – **35px**

Gala rezultāta aptuvenais izskats (tam jābūt mainīgam katrā koda izpildes reizē):



Mājas Darbs Nr. 2

Mājas darbs Nr. 2

Nosacījumi:

1. Programmu izveidē drīkst būt tikai kodi, ko esam apskatījuši šajās nodaļās. Jebkurš cits kods tiks uzskatīts par kļūdu un **tiks samazināts punktu skaits**. Darba izveidei pilnībā pietiek ar kodiem, ko esam apskatījuši, papildus nekas nav vajadzīgs. (Piemēram, nelietot masīvus)
2. ChatGPT Tu vari lietot, lai palīdzētu apgūt JavaScript. Bet mājas darba izveidē ChatGPT lietot **nedrīkst**. Ja mūsu sistēma uzrādīs ka darbu vai tā daļu veicis ChatGPT, **darbs var tikt novērtēts ar 0 vai arī tiks samazināts punktu skaits**.

Izveidojiet jaunu programmu mājas lapā <https://s2js.com/>

Svarīgi ir izveidot jaunu programmu ar citu nosaukumu, jo, ja otro mājas darbu veiksi turpat, kur veidojāt pirmo, rakstīsiet tam pa virsu, tad pirmais darbs būs pazudis.

Mājas darbam ir 3 varianti, kurš ir Tavs – atkarīgs no **Vārda** pirmā burtā.

Ieteikums: Ja sāc pildīt mājas darbu un kautkas neizdodas, raksti datorikas skolotājam. Apraksti kas nesanāk un pievieno jau izdarītā darba saiti (linku)

1. variants

1. var. - Ja Tev **vārds** sākas ar burtiem **A, Ā, B, C, Č, D, E, Ē, F, G, Ģ,**

tad izveidojiet **Canvas**(audeklu) – krāsa - attēlā, ar izmēru 456x456 (2p)

Saite uz suņa attēlu - <https://icons.iconarchive.com/icons/google/noto-emoji-animals-nature/96/22215-dog-icon.png>

1. (5p) Izveidot kodu, kur attēls ar suni (saite/links dots augstāk) atrodas Canvas augšā.

Sākuma pozīcijas **X** koordinātu noteikt ar Random skaitļu ģeneratoru.

Sākuma pozīcijai jābūt tādai, lai neviena attēla daļa abās pusēs neizietu ārpus Canvas.

(Izmantojiet mācību sadaļā esošo kodu, to palabojot)

2. (5p) Lietojot setInterval ar atjaunošanas laiku – 30, likt attēlam virzīties uz leju.

Attēla virzīšanās ātrumu uzlikt tik lēnu, lai ir vismaz 15 sekundes, kamēr attēls sasniedz apakšu.

3. (4p) Kad attēls sasniedz apakšu (attēla apakšējā mala pieskaras Canvas apakšai), tas apstājas.

4. (5p) Pievienot kodam rindiņas, kuras ļauj suņa attēlu pabīdīt pa labi vai pa kreisi ar klaviatūras taustiņiem – virzienbultām.

Katrs taustiņa nospiediens – pabīdīt attēlu par 7 vienībām.

Jāievēro nosacījums – attēls nedrīkst iziet ārpus Canvas. (Izmantojiet mācību sadaļā esošo kodu, to palabojot)

5. (4p) Canavas vidū no vienas malas līdz otrai uzrakstīt savu vārdu. Ar pelēku vai sudraba krāsu. Teksta augstumu lietot pēc vajadzības. Attēlam ar suni ir jāpārvietojas aiz teksta.

Kur nepieciešams, lietot attēla un Canvas augstuma un platuma noteikšanu, nerakstīt konkrētus skaitļus.

Gala rezultāta aptuvenais izskats (attēla novietojums katru koda izpildes reizi ir citā vietā):



2. variants

2. var. - Ja Tev vārds sākas ar burtiem H,I,Ī,J,K,Ķ,L,Ļ,M,N,Ņ

tad izveidojiet **Canvas** (audeklu) – krāsa - attēlā, ar izmēru 512x512 (2p)

Saite uz vāveres attēlu - <https://icons.iconarchive.com/icons/majdi-khawaja/ice-age-4/72/Scrat-2-icon.png>

1. (5p) Izveidot kodu, kur attēls ar vāveri (saite/links dots augstāk) atrodas Canvas augšā.

Sākuma pozīcijas **X** koordinātu noteikt ar Random skaitļu ģeneratoru.

Sākuma pozīcijai jābūt tādai, lai neviena attēla daļa abās pusēs neizietu ārpus Canvas.

(Izmantojiet mācību sadaļā esošo kodu, to palabojot)

2. (5p) Lietojot setInterval ar atjaunošanas laiku – 35, likt attēlam virzīties uz leju.

Attēla virziņā ātrumu uzlikt tik lēnu, lai ir vismaz 20 sekundes, kamēr attēls sasniedz apakšu.

3. (4p) Kad attēls sasniedz apakšu (attēla apakšējā mala pieskaras Canvas apakšai), tas apstājas.

4. (5p) Pievienot kodam rindiņas, kuras ļauj vāveres attēlu pabīdīt pa labi vai pa kreisi ar klaviatūras taustiņiem – virzienbultām.

Katrs taustiņa nospiediens – pabīdīt attēlu par 9 vienībām.

Jāievēro nosacījums – attēls nedrīkst iziet ārpus Canvas. (Izmantojiet mācību sadaļā esošo kodu, to palabojot)

5. (4p) Canavas vidū no vienas malas līdz otrai uzrakstīt savu vārdu. Ar sarkanu krāsu.

Teksta augstumu lietot pēc vajadzības. Attēlam ar vāveri ir jāpārvietojas aiz teksta.

Kur nepieciešams, lietot attēla un Canvas augstuma un platuma noteikšanu, nerakstīt konkrētus skaitļus.

Gala rezultāta aptuvenais izskats (attēla novietojums katru koda izpildes reizi ir citā vietā):



3. variants

3. var. - Ja Tev **vārds** sākas ar burtiem **O,P,R,S,Š,T,U,Ū,V,Z,Ž**

tad izveidojiet **Canvas** (audeklu) – krāsa - attēlā, ar izmēru 493x493 (2p)

Saite uz automašīnas attēlu - <https://icons.iconarchive.com/icons/cemagraphics/classic-cars/72/vw-beetle-icon.png>

1. (5p) Izveidot kodu, kur attēls ar auto (saite/links dots augstāk) atrodas Canvas augšā.
Sākuma pozīcijas **X** koordinātu noteikt ar Random skaitļu ģeneratoru.
Sākuma pozīcijai jābūt tādai, lai neviena attēla daļa abās pusēs neizietu ārpus Canvas.
(Izmantojiet mācību sadaļā esošo kodu, to palabojot)
2. (5p) Lietojot setInterval ar atjaunošanas laiku – 40, likt attēlam virzīties uz leju.
Attēla virzīšanās ātrumu uzlikt tik lēnu, lai ir vismaz 20 sekundes, kamēr attēls sasniedz apakšu.
3. (4p) Kad attēls sasniedz apakšu (attēla apakšējā mala pieskaras Canvas apakšai), tas apstājas.
4. (5p) Pievienot kodam rindiņas, kuras ļauj auto attēlu pabīdīt pa labi vai pa kreisi ar klaviatūras taustiņiem – virzienbultām.
Katrs taustiņa nospiediens – pabīdīt attēlu par 11 vienībām.
Jāievēro nosacījums – attēls nedrīkst iziet ārpus Canvas. (Izmantojiet mācību sadaļā esošo kodu, to palabojot)
5. (4p) Canvas vidū no vienas malas līdz otrai uzrakstīt savu vārdu. Ar zilu krāsu.
Teksta augstumu lietot pēc vajadzības. Attēlam ar auto ir jāpārvietojas priekšā (virs) tekstam.

Kur nepieciešams, lietot attēla un Canvas augstuma un platuma noteikšanu, nerakstīt konkrētus skaitļus.

Gala rezultāta aptuvenais izskats (attēla novietojums katru koda izpildes reizi ir citā vietā):

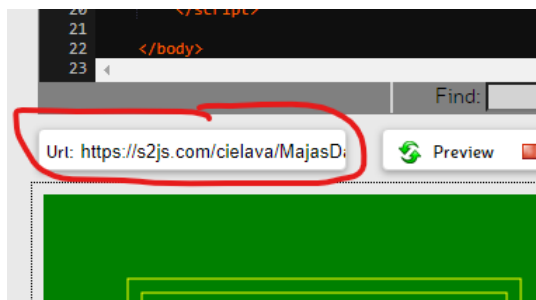
Gatis Cid¹ava



Uzdevumu iesniegšana un nosacījumi

Uzdevumu iesniegšana un nosacījumi

Nokopējiet iegūtā darba saiti (linku), skatīt attēlā. Šo saiti Jums būs jāpievieno ieskaitei tās pildīšanas laikā. Kopējot aizvelc peli vairāk pa labi, jo daļa no saites var būt paslēpusies.

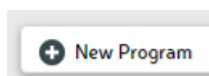


Visa saite izskatās šādi :

<https://s2js.com/Tavs lietotāja vārds/Programmas nosaukums>.

Piemēram: <https://s2js.com/saule11/majasdarbs81>

Ja izveidoji mājas darbu (programmu) ar kaut kādu nosaukumu, tad taisot otru mājas darbu, Tev jāizvēlas



jauna programma un cits nosaukums, jo, ja turpināsi darbu šajā pašā programmā, sabojāsi jau izveidoto mājas darbu.

Pirms iesniegšanas atver sava darba saiti un pārliecinies ka Tavs darbs ir redzams.

Nosacījumi:

1. Programmu izveidē drīkst būt tikai kodi, ko esam apskatījuši šajās nodaļās. Jebkurš cits kods tiks uzskatīts par kļūdu un **tiks samazināts punktu skaits**. Darba izveidei pilnībā pietiek ar kodiem, ko esam apskatījuši, papildus nekas nav vajadzīgs. (Piemēram, nelietot masīvus)
2. ChatGPT Tu vari lietot, lai palīdzētu apgūt JavaScript. Bet mājas darba izveidē ChatGPT lietot **nedrīkst**. Ja mūsu sistēma uzrādīs ka darbu vai tā daļu veicis ChatGPT, **darbs var tikt novērtēts ar 0 vai arī tiks samazināts punktu skaits**.

Noslēguma tests

👉 Izpildi kopsavilkuma pārbaudes testu par 1.ieskaites mācību materiālu. Vērtēšanas kritēriji: par kopsavilkuma pārbaudes testu var saņemt 50 punktus.

Temata apguve

🧠 **Apsveicam! Tu esi apguvis 1. ieskaites tēmu kursā “Datorika”!** 📖

Šis macību materiāls “**Javascript (2. daļa no 5)**” ļāva turpināt JavaScript apguvi.

🇸🇻 **Šajā mācību materiālā Tu:**

- 1 Uzzināji, kas ir Random skaitļi un kā tos izmantot
- 2 Saprati, kas ir SetInterval (funkciju izsaukšanai ik pēc noteikta laika)
- 3 Apguvi attēla ienešanu un attēlošanu uz Canvas
- 4 Uzzināji, kā likt Tavam kodam reaģēt uz klaviatūras taustiņu nospiešanu
- 5 Izveidoji 2. mājas darbus, tie būs jāpievieno ieskaitei

🔗 **Kā iegūtās zināšanas Tev noderēs dzīvē?**

- ✓ **Ikdienā** Zināsi no kā sastāv mājaslapas un ka to izveidošana prasa nopietnas zināšanas.
- ✓ **Mācībās** Noderēs spēja risināt problēmas un domāt loģiski.
- ✓ **Nākotnē** zināšanas programmēšanā var Tev dot iespēju strādāt ļoti labi apmaksātā darbā.

Tagad pārdomā apgūto un atbildi uz pašrefleksijas jautājumiem! ✓